

Restaurant POS Backend API List

Base URL: <http://localhost:8080> | Generated: May 09, 2026 | Total endpoints listed: 63

Authentication uses a JWT Bearer token returned from `/api/auth/login`. Use Authorization: Bearer for protected endpoints. Spring roles are checked with `ROLE_` internally, but this document lists the role names as they appear in the app.

Role Group	Members / Meaning
STAFF	ADMIN, WAITER, CASHIER
ORDERING	ADMIN, WAITER, CASHIER, CUSTOMER
KITCHEN	ADMIN, WAITER, CASHIER, KITCHEN
Admin-only	ADMIN

Enum	Allowed Values
Role	ADMIN, WAITER, CASHIER, KITCHEN, CUSTOMER
OrderType	DINING, TO_GO, DELIVERY
OrderStatus	SERVING, COMPLETED, CANCELLED
PaymentStatus	UNPAID, PAID
TransactionMethod	NONE, CASH, CARD, SPLIT
CardType	NONE, VISA, MASTERCARD, AMEX, DISCOVER, OTHERS
TableStatus	AVAILABLE, OCCUPIED, RESERVED
ModifierType	ADD, NO, SWITCH

Authentication

Method	Path	Auth / Roles	Request	Purpose / Response
POST	/api/auth/login	Public	Body: LoginRequest { username, password }	Returns LoginResponse { id, username, role, tableId, tableLabel, tableSeat, token }.

Tables

Method	Path	Auth / Roles	Request	Purpose / Response
GET	/api/table	ADMIN, WAITER, CASHIER	None	List all restaurant tables.
GET	/api/table/{id}	ADMIN, WAITER, CASHIER, CUSTOMER	Path: id	Get one table. Customer may use this for their assigned table.
POST	/api/table	ADMIN, WAITER, CASHIER	Body: RestaurantTable { label, seat, tableStatus, posX, posY }	Create a table.
PUT	/api/table/{id}?userId={userId}	ADMIN, WAITER, CASHIER	Path: id; Query: userId; Body: RestaurantTable	Update label, seats, status, or saved layout position.
DELETE	/api/table/{id}	ADMIN, WAITER, CASHIER	Path: id	Delete table.

Users And Table Assignment

Method	Path	Auth / Roles	Request	Purpose / Response
GET	/api/user	ADMIN	None	List all users.
GET	/api/user/customers	ADMIN, WAITER, CASHIER	None	List customer accounts with table assignment summaries.
PUT	/api/user/table-assignments/{tableId}	ADMIN, WAITER, CASHIER	Path: tableId; Body: { customerId }	Assign one customer account to a table or clear assignment with null customerId.
POST	/api/user	ADMIN	Body: CreateUserRequest { username, password, role, tableId }	Create user with hashed password.
PUT	/api/user/{id}	ADMIN	Path: id; Body: User	Update a user.
DELETE	/api/user/{id}	ADMIN	Path: id	Delete a user.

Orders

Method	Path	Auth / Roles	Request	Purpose / Response
GET	/api/order	ADMIN, WAITER, CASHIER	Optional query: date, status, type	List orders. Supports date/status/type filtering.
GET	/api/order/{id}	ADMIN, WAITER, CASHIER	Path: id	Get full order with table, customer, and cart items.
POST	/api/order	ADMIN, WAITER, CASHIER, CUSTOMER	Body: CreateOrderRequest { tableId, userId, orderType, customer }	Create order directly.
POST	/api/order/tables/{tableId}/open?userId={userId}	ADMIN, WAITER, CASHIER, CUSTOMER	Path: tableId; Query: userId	Create or reuse current serving order for a table. Customer must match assigned table.
POST	/api/order/tables/{tableId}/close	ADMIN, WAITER, CASHIER	Path: tableId; Body: CloseOrderRequest { code }	Force delete active table order and free table. Code checked on backend.
PUT	/api/order/{id}	ADMIN, WAITER, CASHIER	Body: UpdateOrderRequest	Update order type/status/payment/method/card/totals.
PUT	/api/order/{id}/customer	ADMIN, WAITER, CASHIER	Body: UpdateOrderCustomerRequest { name, address, phoneNumber, note }	Update or attach customer info for an order.
PUT	/api/order/{id}/move-table/{newTableId}	ADMIN, WAITER, CASHIER	Path: id, newTableId	Move order to another table and update table occupancy.
PUT	/api/order/{id}/checkout	ADMIN, WAITER, CASHIER	Body: CheckoutRequest { transactionMethod, cardType, subtotal, tips, tax, total }	Mark order completed/paid and free dining table.
DELETE	/api/order/{id}	ADMIN, WAITER, CASHIER	Path: id	Delete order; frees table if it was a dining table order.

Cart Items And Notes

Method	Path	Auth / Roles	Request	Purpose / Response
GET	/api/cart	ADMIN, WAITER, CASHIER, KITCHEN	None	List all cart items.
GET	/api/cart/notFinished	ADMIN, WAITER, CASHIER, KITCHEN	None	Kitchen queue: sent items that are not finished.
GET	/api/cart/order/{orderId}/notFinished	ADMIN, WAITER, CASHIER, KITCHEN	Path: orderId	Kitchen queue for one order.
GET	/api/cart/{id}	ADMIN, WAITER, CASHIER, CUSTOMER	Path: id	Get cart item by id.
GET	/api/cart/order/{orderId}	ADMIN, WAITER, CASHIER, CUSTOMER	Path: orderId	Get cart items for an order.
POST	/api/cart	ADMIN, WAITER, CASHIER, CUSTOMER	Body: CartItem { orderId, menuItemId, quantity }	Add item to order. Backend snapshots menu name and price.
PUT	/api/cart/{id}	ADMIN, WAITER, CASHIER, CUSTOMER	Path: id; Body: CartItem	Update cart item, commonly quantity.

Method	Path	Auth / Roles	Request	Purpose / Response
PUT	/api/cart/{id}/send	ADMIN, WAITER, CASHIER, CUSTOMER	Path: id	Move pending item to sent/preparing state.
PUT	/api/cart/{id}/finish	ADMIN, WAITER, CASHIER, KITCHEN	Path: id	Mark kitchen item finished.
PUT	/api/cart/{id}/revert-finish	ADMIN, WAITER, CASHIER, KITCHEN	Path: id	Move item back from finished to preparing.
DELETE	/api/cart/{id}	ADMIN, WAITER, CASHIER, CUSTOMER	Path: id	Delete cart item and recalculate order total.
GET	/api/cart/{cartItemId}/notes	ADMIN, WAITER, CASHIER	Path: cartItemId	List notes for one cart item.
POST	/api/cart/{cartItemId}/notes	ADMIN, WAITER, CASHIER	Path: cartItemId; Body: CartItemNote { note, price }	Create staff-only item note/addition.
PUT	/api/cart/notes/{noteId}	ADMIN, WAITER, CASHIER	Path: noteId; Body: CartItemNote { note, price }	Update item note/addition.
DELETE	/api/cart/notes/{noteId}	ADMIN, WAITER, CASHIER	Path: noteId	Delete item note/addition.

Menu Items

Method	Path	Auth / Roles	Request	Purpose / Response
GET	/api/menuitem	Authenticated	None	List menu items with modifiers.
GET	/api/menuitem/{id}	Authenticated	Path: id	Get one menu item.
POST	/api/menuitem	ADMIN, WAITER, CASHIER	Body: MenuItem { name, price, isAvailable, categories? }	Create menu item.
PUT	/api/menuitem/{id}	ADMIN, WAITER, CASHIER	Path: id; Body: MenuItem	Update menu item price/name/availability.
DELETE	/api/menuitem/{id}	ADMIN, WAITER, CASHIER	Path: id	Delete menu item. Cart snapshots protect existing orders.

Menu Item Modifiers

Method	Path	Auth / Roles	Request	Purpose / Response
GET	/api/menuitemmodifier	ADMIN, WAITER, CASHIER	None	List all modifiers.
GET	/api/menuitemmodifier/{id}	ADMIN, WAITER, CASHIER	Path: id	Get modifier by id.
GET	/api/menuitemmodifier/menuitem/{menuItemId}	ADMIN, WAITER, CASHIER	Path: menuItemId	List modifiers for one menu item.
GET	/api/menuitemmodifier/menuitem/{menuItemId}/type/{modifierType}	ADMIN, WAITER, CASHIER	Path: menuItemId, modifierType	List modifiers by type: ADD, NO, SWITCH.
POST	/api/menuitemmodifier	ADMIN, WAITER, CASHIER	Body: MenuItemModifierRequest { menuItemId, modifierType, name, switchTo }	Create modifier.
PUT	/api/menuitemmodifier/{id}	ADMIN, WAITER, CASHIER	Path: id; Body: MenuItemModifierRequest	Update modifier.
DELETE	/api/menuitemmodifier/{id}	ADMIN, WAITER, CASHIER	Path: id	Delete one modifier.
DELETE	/api/menuitemmodifier/menuitem/{menuItemId}	ADMIN, WAITER, CASHIER	Path: menuItemId	Delete all modifiers for a menu item.

Categories

Method	Path	Auth / Roles	Request	Purpose / Response
GET	/api/category	ADMIN, WAITER, CASHIER	None	List categories.
GET	/api/category/{id}	ADMIN, WAITER, CASHIER	Path: id	Get category by id.
POST	/api/category	ADMIN, WAITER, CASHIER	Body: Category { name, menuitems? }	Create category.
PUT	/api/category/{id}	ADMIN, WAITER, CASHIER	Path: id; Body: Category	Update category.
DELETE	/api/category/{id}	ADMIN, WAITER, CASHIER	Path: id	Delete category.

Customers

Method	Path	Auth / Roles	Request	Purpose / Response
GET	/api/customer	ADMIN, WAITER, CASHIER	None	List customer info records.
GET	/api/customer/{id}	ADMIN, WAITER, CASHIER	Path: id	Get customer by id.
GET	/api/customer/phone/{phone}	ADMIN, WAITER, CASHIER	Path: phone	Lookup customer by phone.
GET	/api/customer/name/{name}	ADMIN, WAITER, CASHIER	Path: name	Lookup customer by name.
GET	/api/customer/address/{address}	ADMIN, WAITER, CASHIER	Path: address	Lookup customer by address.
POST	/api/customer	ADMIN, WAITER, CASHIER	Body: Customer { name, address, phoneNumber, note }	Create customer info record.
PUT	/api/customer/{id}	ADMIN, WAITER, CASHIER	Path: id; Body: Customer	Update customer info record.
DELETE	/api/customer/{id}	ADMIN, WAITER, CASHIER	Path: id	Delete customer info record.

Common Request Body Shapes

Name	Fields
LoginRequest	username, password
LoginResponse	id, username, role, tableId, tableLabel, tableSeat, token
CreateUserRequest	username, password, role, tableId
RestaurantTable	id, label, seat, tableStatus, posX, posY
CreateOrderRequest	tableId, userId, orderType, customer
UpdateOrderRequest	orderType, orderStatus, paymentStatus, transactionMethod, cardType, subtotal, tips, tax, total
CheckoutRequest	transactionMethod, cardType, subtotal, tips, tax, total
CloseOrderRequest	code
CartItem	id, orderId, menuItemId, name, price, quantity, isPending, isFinished, sentAt, finishedAt, notes
CartItemNote	id, cartItemId, note, price, createdAt
MenuItem	id, name, price, isAvailable, modifiers, createdAt, updatedAt
MenuItemModifierRequest	menuItemId, modifierType, name, switchTo
Customer	id, name, address, phoneNumber, note

Important Backend Rules

Rule	Behavior
Close code	Stored in backend application.properties as app.order-close-code=4900; frontend only sends user input.
Customer table restriction	Customer can open only their assigned table; other table opens return an error.
Table release	Checkout, delete order, close order, or switching dining order to non-dining frees a table.
Kitchen finish	Marking an item finished does not free the table or pay the order.
Item snapshots	Cart items store name_snapshot and price_snapshot so menu edits do not rewrite order item prices.
Note pricing	Each cart item can have many CartItemNote rows; each note may have price 0 or a positive/negative adjustment.